files in a filesystem structure that maps to namespaces and sub-namespaces. Multiple `namespace` statements in a single file violates section 3 of the PHP-FIG PSR-12 coding standards. Don't get into the habit of using multiple namespaces like this!

With that said, let's examine the code. The keyword `namespace` followed (optionally) by a legal name (same rules as variable and function naming) denotes the following code is using that namespace. Using no name at all simply means the code is not in any particular namespace, so it uses the root namespace. The opening and closing braces that follow the declaration aren't necessary but are highly recommended for readability. Incidentally, the root namespace is often referred to as the "global" namespace, but this is inaccurate: things defined in root are not magically available globally, as we shall see. Also, using the word "global" might cause confusion with the concept of global scope. Scope and namespace are not the same thing, which we'll also be covering in a moment.

Now let's see how we use each namespaced version of `collision()`. We have a few options.

Calling Functions from Namespaces

```
namespace {
    collision();
}

namespace Other {
    collision();
}

namespace Other\Sub {
    collision();
}

Output:
Hello from root namespace.
Hello from Other namespace.
Hello from Other\\Sub namespace.
```

Here we are entering each namespace and calling `collision()`. The PHP engine will search whatever its current namespace is for the function. As we have defined `collision()` in each, we get the version specific to that namespace, which we can see in